

Decoupling Workload Admission from Policy Materialization in Kubernetes

*AI-generated working draft for author discussion
Non-submission exercise; Traditional Research framing*

Editorial notice. This is an AI-generated writing exercise, not a submission or text attributed to Antonio Ojea. It develops the proposed research argument using the available evidence. Implementation inspection, archived measurements, operator-reported outcomes, and analytical predictions are distinguished throughout. The evaluation is incomplete.

Abstract

Network policy enforcement must keep pace with the creation and replacement of the workloads it protects. Indexed kernel lookups and incremental datapath updates improve packet processing, but they do not by themselves remove the cost of translating changing workload metadata into distributed enforcement state. We examine a different placement of that work: evaluate Kubernetes policy semantics in userspace when communication is attempted, and reuse accepted decisions through the kernel’s connection tracker. The kube-network-policies implementation combines NFQUEUE-based admission with nftables packet steering; runtime integration provides local Pod metadata without waiting for its asynchronous publication through the Kubernetes API. We characterize the design using source inspection and archived experiments targeting 35,000 Pods on a reported 720-worker testbed. Completed Cilium identity sweeps retain low post-scheduling latency and flat worker CPU (0.51–0.63 mean cores) across 7 to 35,000 intended identities, while the 35,000-ReplicaSet tiers halve achieved Pod throughput (170 versus 280–330 Pods/s) and a dense-mesh tier was excluded because its estimated per-endpoint permission state exceeded the stated map capacity. An earlier Cilium release stranded Pods at 700 identities, and a separate direct-burst experiment stranded 5,694 Pods. These observations motivate distinguishing latency, throughput, capacity, and admission failure, rather than treating identity cardinality as a single scaling variable. We identify the conditions under which deferred evaluation could reduce eager policy state, the new connection-processing and fail-open risks it introduces, and we enumerate the measurements absent from the artifacts—including any matched kube-network-policies identity sweep—that are

needed to establish its comparative benefit.

1 Introduction

The unit of isolation in a container cluster is also a unit of change. A service scales out, a job completes, or a sandbox is replaced; its network permissions must follow that lifecycle. For long-lived services, much of this work can be amortized over hours of communication. Short-lived workloads offer less time to recover their setup cost. Agent execution environments provide one motivation for this regime, alongside batch jobs and serverless functions, although the experiments considered here are synthetic sandbox workloads rather than traces of deployed agents.

Kubernetes makes the problem more subtle than installing a firewall rule for a new address. NetworkPolicies describe permitted relationships through Pod labels, namespace labels, ports, and address ranges. Addresses and placement change independently of this intent. An implementation must join policy with current workload metadata, establish isolation at the appropriate point in the workload lifecycle, and handle updates while packets continue to arrive. A fast packet lookup is necessary in many settings, but it is not a complete answer to these coordination costs.

Successive enforcement mechanisms have addressed different parts of this problem. Linear chains give way to indexed sets and maps. Programmable datapaths separate packet-processing code from mutable state. Distributed network virtualization compiles logical intent into local actions and caches expensive decisions. These mechanisms are complementary, not a sequence of obsolete technologies. In particular, nftables supports incremental updates, and userspace miss handling followed by kernel caching is central to Open vSwitch [9, 26]. The next question is therefore not which kernel mechanism is newest. It is which work must occur before a workload can communicate, and how much of that work must be repeated elsewhere in the cluster.

This paper examines *deferred semantic evaluation* for Kubernetes network policy. In kube-network-policies, hereafter

KNP, a node agent receives packets selected by nftables through NFQUEUE, resolves the relevant metadata, and evaluates the policy in userspace. An accepted verdict records an acceptance label in conntrack. Eligible subsequent traffic follows the kernel path without repeating the semantic evaluation. The engine does not require a separate policy-specific distributed security-identity allocator or a per-endpoint expansion of every permitted peer identity. It still requires metadata distribution, packet steering, and per-connection state.

The design trades eager permission materialization for work at attempted communication. If a policy permits a large peer set but a workload contacts only a few peers, maintaining every possible relationship may be unnecessary. Conversely, a workload creating many short connections can make the queue and userspace evaluator the bottleneck. Denied traffic is especially important: repeatedly rejected attempts do not benefit from the accepted connection cache. The research question is whether this tradeoff is favorable for high-churn clusters, not whether userspace processing is universally faster than in-kernel processing.

Local startup observation is a second part of the design. Kubernetes publishes a Pod’s address asynchronously. A policy agent relying exclusively on that publication can learn the address after a container has started using it, or after a lifecycle hook has begun waiting for network access. The longstanding issue documented in Kubernetes issue 85966 illustrates this dependency [11]. Node Resource Interface (NRI) integration obtains local sandbox metadata through the runtime, removing that particular round trip from local address discovery. It does not eliminate namespace or policy convergence, nor establish a cluster-wide startup barrier.

Our evidence distinguishes four observations. First, the completed Cilium identity sweeps do not show an increasing post-scheduling latency curve: the intended 35,000-identity open-gateway configuration has a 1.99-second P99, and mean worker CPU stays within 0.51–0.63 cores across all tiers. Second, the 35,000-ReplicaSet tiers reduce achieved Phase-1 throughput from roughly 280–330 to 170 Pods/s and raise cluster-scoped Pod LIST P99 latency from 14–19 s to 28.8 s; the controller fan-out, not the datapath, is the visible bottleneck there. Third, under the benchmark’s per-endpoint expansion model, a bidirectional mesh with 35,000 identities requires approximately 70,000 peer-policy entries per endpoint. The experiment producer excluded that tier on predicted capacity grounds; it was not executed. Fourth, two admission failures are recorded: an archived Cilium v1.18.6 run stranded 19 of 35,000 Pods at 700 identities, attributed to a policy-cache race fixed in v1.20.0 [1], and a v1.20.1 raw-Pod burst aborted with 29,306 Pods Running and 5,694 stranded, attributed to endpoint throttling and cumulative identity allocation. None of these observations should be substituted for another.

The paper develops three contributions at different levels of evidence. It describes a working enforcement decomposition based on existing Linux mechanisms, including a se-

curity analysis of its fail-open and unknown-peer behavior; presents a conditional model of eager permission state versus connection-driven work; and analyzes an archived large-cluster benchmark collection, including its limitations and incomplete outcomes. Section 6.7 enumerates what the artifacts do not contain. In particular, no matched KNP identity sweep exists: every archived KNP run uses six ReplicaSets, so the paper does *not* establish superior identity scaling for KNP, and it quantifies NFQUEUE admission overhead only through an indicative three-node microbenchmark. Making those limits explicit lets the evaluation test the design rather than assuming its conclusion.

2 Background and Problem

2.1 Policy Semantics and Workload Lifecycle

Kubernetes NetworkPolicy isolates ingress and egress independently. Within a direction, the permissions of applicable standard NetworkPolicies combine additively. A connection between two isolated Pods must satisfy the source’s egress policy and the destination’s ingress policy; reply traffic for an allowed connection is implicitly permitted [19]. This connection-oriented interpretation makes reuse of admission decisions natural, although it does not require any particular conntrack implementation.

Policy creation and workload creation have different synchronization requirements. A newly submitted policy is handled asynchronously, and the API does not expose a general enforcement-completion barrier. Once a plugin has handled a policy, newly created selected Pods must be isolated before their containers start. Allow permissions may converge later. Thus a system can preserve isolation while temporarily delaying legitimate communication. Safety and availability must be measured separately; a fast transition to Running is not proof of either.

We distinguish four quantities that are often conflated: live endpoints, distinct policy-relevant label sets, allocated numeric security identities, and materialized datapath entries. Several Pods can share an identity. One identity can contribute permissions to many endpoint maps. Allocations not yet reclaimed can outlive their original workload. The number of permitted relationships also differs from the number of active connections. The resulting scaling dimensions are related but not interchangeable.

2.2 Placement and Timing of Enforcement Work

Chains, sets, and incremental updates. An ordered chain can perform work proportional to the number of rules visited. This does not imply that every iptables deployment performs a linear address lookup: ipset supplies indexed membership, while nftables integrates sets and verdict maps. Sutter’s 2017

measurements demonstrate the importance of this representation choice, with both nftables sets and iptables with ipset maintaining throughput across the tested ranges [12]. More recent Kubernetes work describes incremental nftables updates that scale with changed Service state rather than the full Service ruleset [26]. Service proxying is not NetworkPolicy, but it demonstrates why lookup and update mechanisms must be discussed independently.

Virtual networking and reactive caches. NVP and Open vSwitch address the translation from logical networks to physical forwarding [4, 9]. OVS caches flow-processing results and sends misses to userspace; its evolution illustrates both the value and the cost of reactive processing. OVN also supports stateful ACLs, including *allow-related* actions backed by connection tracking [23]. A comparison that describes these systems as inherently stateless would miss the closest precedents for our design.

Programmable datapaths and identities. eBPF maps permit userspace to update keyed state without replacing the packet-processing program [20]. Cilium uses label-derived security identities to represent workloads [15]. Identity sharing can avoid repeating work for equivalent workloads, but distinct labels and broad selectors can still create allocation, distribution, and materialization work. These are properties of an architecture and configuration, not intrinsic restrictions of eBPF. The size of a numeric identity space and the capacity of a policy map are different limits.

Problem statement. We ask whether policy enforcement can retain the established kernel forwarding path while avoiding eager expansion of all permitted peers at each endpoint. A useful solution must explain where the displaced work goes, how local admission obtains sufficiently fresh metadata, and how existing decisions interact with changing policy. Packet throughput alone cannot answer these questions.

3 Design

3.1 Connection Admission and Cached Decisions

KNP separates policy interpretation from packet steering. Kubernetes objects supply policy intent and workload metadata to the node agent. nftables identifies traffic requiring a decision and sends it to NFQUEUE. The callback parses packet headers, invokes the userspace evaluator, and returns an accept or drop verdict. For an accepted decision it supplies a conntrack label, which the kernel rules can subsequently recognize. Figure 1 illustrates the separation.

The cached decision is connection state, not a precomputed table of every allowed peer. In the inspected forwarding path,

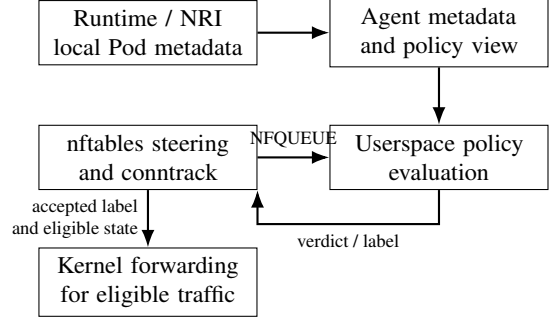


Figure 1: Enforcement decomposition, omitting protocol and host-traffic exceptions. Kubernetes watches also feed the agent’s policy and metadata view. NRI supplies local observations, not remote policy synchronization.

the bypass rule requires both the acceptance label and an established or related conntrack state. Uncached managed traffic is queued. Consequently, “first-packet evaluation” is a useful intuition but not an exact accounting rule: several initial packets, retransmissions, or protocol-specific transitions can reach userspace before the bypass condition holds. Denied packets do not acquire the accepted-decision cache entry.

nftables remains part of the implementation. It maintains steering rules and managed-address sets, rather than executing Kubernetes label selectors as a shared kernel policy table. Avoiding peer-permission expansion does not imply eliminating every local rule update. Changes to managed endpoints or enforcement scope can still require steering reconciliation. The policy evaluator’s own metadata indexes and selector processing also consume CPU and memory.

The decision is local once the required metadata has arrived: the packet path does not allocate a new distributed numeric identity for the attempted connection. This is a receive-driven policy metadata path, not a claim that the entire network control plane is read-only. The runtime, address management, routing, and Kubernetes controllers continue to update their own state. Remote endpoint metadata must still reach the enforcing node.

3.2 Runtime Metadata and Startup Isolation

Consider the dependency chain when a node learns its own Pod’s IP only through Kubernetes status. The runtime creates the sandbox and configures networking; the kubelet publishes status to the API server; a watch event reaches the policy agent. Container or lifecycle-hook activity can occur before the last step. If that activity waits for network permissions that depend on the status update, observation delay becomes application delay. Issue 85966 records precisely this problem for policy-only networking [11].

The NRI resolver populates a local IP-to-Pod metadata cache in its `RunPodSandbox` callback. It uses addresses sup-

plied by the runtime API and can fall back to inspection of the sandbox network namespace. This places local observation at the runtime boundary instead of behind the kubelet-API-server-watch round trip. Subject to the runtime’s callback ordering and successful metadata acquisition, it makes the local address available before application-container execution requires it.

There are two limits to this improvement. First, obtaining metadata is not the same as installing effective isolation. The integration must ensure the steering path cannot bypass policy before protection applies. Second, namespace labels, policy objects, and remote peers can still be asynchronous. A claim of zero startup latency would therefore be incorrect. The narrower claim is removal of a specific asynchronous dependency in local address discovery; an NRI-on/off timing and connectivity experiment is needed to quantify its benefit.

3.3 Policy Changes and Failure Semantics

Caching makes policy changes part of the correctness argument. Standard NetworkPolicy leaves the effect of updates on already established connections implementation-defined [19]. KNP also provides a strict mode that scans conntrack entries, reevaluates supported flows, and clears the acceptance label where current policy no longer permits communication. Subsequent processing can then revisit the decision. This introduces work proportional to the flows inspected and the complexity of reevaluation; it is not an instantaneous revocation mechanism. The enforcement delay must include event delivery, scan scheduling, scan execution, and subsequent packet handling.

Failure policy must be equally explicit. NFQUEUE can drop packets on queue overflow or accept them when configured fail-open [21]. The inspected command-line interface defaults to fail-open, and the dataplane sets the queue’s bypass flag whenever fail-open is enabled, with a maximum queue length of 1,024 packets. Two facts about the cached path make this a security-relevant default rather than an operational detail. Denied packets never acquire the acceptance label, so every retried forbidden connection attempt re-enters the queue; and a workload that can emit uncached packets faster than the agent returns verdicts fills the queue. Under fail-open, the overflow is then *accepted* by the kernel without evaluation. A tenant that is permitted to run code in the cluster can therefore attempt to convert denied traffic into a bypass by saturating its node’s agent, and its own denied attempts are the cheapest saturating traffic. The inspected implementation has no per-source rate limit or denied-flow negative cache that would blunt this amplification; the only countermeasure is `-fail-open=false`, which converts overload into loss of connectivity for every managed Pod on the node. Quantifying the overload point is therefore part of the safety argument, not only of the performance argument. The benchmark manifest in the repository sets `-nfqueue-id` but not `-fail-open`, so

the archived KNP runs executed with the fail-open default.

Unknown peers have a different failure mode. When the evaluator cannot resolve a remote source or destination address to Pod metadata, the inspected selector matching treats a `nil` peer as matching only `ipBlock` rules; Pod and namespace selectors never match it. Under a default-deny policy the packet is dropped. This is fail-closed at the policy level, and it is the correct isolation behavior, but it means a permitted connection from a freshly created remote Pod is denied until the destination node learns the source’s metadata. NRI removes this delay only for *local* Pods. For remote peers the agent still depends on API-server watch delivery, or on the IPTracker distribution tier of Section 8.1, and the observed delay is an availability cost that the startup benchmarks do not measure.

Further, the inspected implementation logs an initial nftables synchronization error without blocking startup, and its fail-closed graceful-shutdown path removes its nftables rules. Those transitions require a deployment-level isolation contract or implementation changes before claiming protection throughout the lifecycle.

We therefore state the desired safety property conditionally. For a policy already handled by an agent, a new selected workload must not send or receive forbidden traffic before enforcement takes effect. An allowed connection may be delayed by missing information; such delay must not be confused with an isolation violation. The current benchmark collection does not establish this property. A complete validation must exercise missing metadata, label changes, IP reuse, failures, and init containers as well as steady-state allow/deny behavior.

The intended trust boundary includes the node kernel, runtime, policy agent, and the CNI’s endpoint/address binding. Untrusted workloads must not be able to impersonate a peer by spoofing its address. Privileged workloads, host networking, and enforcement around address translation need explicit scope. NFQUEUE and NRI do not independently establish those assumptions.

4 Where the Scaling Cost Moves

Let I denote selected peer identities, P_ℓ local endpoints, d the number of materialized directions, and F_ℓ locally tracked active connections. In an eager representation with a distinct entry per peer identity and direction at every endpoint, the peer-policy contribution is

$$E_{\text{endpoint}} \approx dI, \quad E_{\text{node}} \approx dIP_\ell. \quad (1)$$

This is a conditional representation model, not a lower bound on all policy implementations. Sharing, wildcarding, aggregation, or another representation can change it. Additional policy rules contribute further entries. Entry counts are also not byte counts: data-structure overhead, preallocation, and sharing must be measured separately.

Under the same assumptions, a new identity selected by every local endpoint can induce approximately dP_ℓ local entry

updates. For a rate λ_I of such new identities, the corresponding update demand is approximately $dP_I \lambda_I$. Identity dissemination, selector recomputation, and endpoint regeneration are separate costs; none is measured by this expression. With a shared identity, many Pod replacements may leave this demand nearly unchanged. Fresh-identity churn is consequently a different workload from Pod churn.

Deferred evaluation removes this particular eager cross-product, but retains metadata, steering state, and conntrack. A useful accounting is

$$M_{\text{deferred}} = M_{\text{metadata}} + M_{\text{steering}} + M_{\text{conntrack}}(F_\ell). \quad (2)$$

It is not a proof that total memory is smaller than in an identity-based engine: both may already maintain conntrack, and the userspace metadata representation can dominate. The avoided term is materialized permission state; the comparison must account for all other state on both sides.

Let λ_q be the actual queued-packet rate and μ the aggregate sustainable verdict-processing rate. Avoiding sustained queue growth requires, as a first-order condition,

$$\rho = \lambda_q / \mu < 1. \quad (3)$$

Burstiness, shared CPU, and scheduling can produce tail latency and drops even below this average-load condition. Queue demand includes denied attempts and repeated initial packets, not just successfully established connections. The only measurement of μ available to us is the repository’s own three-node ApacheBench microbenchmark [18]: 10,000 non-keepalive HTTP requests at concurrency 1,000 completed at 2,317 requests/s with a 5-ms median and a 3,080-ms P99 dominated by connect time, while the kernel logged `nf_conntrack: table full` at the default 262,144-entry limit. The accompanying Prometheus charts show per-node queued-packet peaks near 8,000 packets/s and callback durations with medians of 80–120 μs and P99s of 170–630 μs . Hardware, kernel, and agent version are not recorded, and conntrack exhaustion confounds the tail, so we treat these as an order-of-magnitude bound on a single node’s verdict capacity, not as a characterization of μ . The 720-node startup reports do not establish λ_q at all.

The favorable regime is thus not simply “many identities.” It is a large or rapidly changing set of possible permissions relative to the communication actually attempted, with sufficient admission-processing capacity. A dense *permission* graph can still produce sparse traffic. A dense *active connection* graph, frequent revocation, or sustained denied traffic can remove that advantage. This distinction supplies testable hypotheses without assuming a particular technology must win.

5 Implementation

The inspected implementation is the Go-based kube-network-policies repository at revision 9984dcd. Its dataplane con-

troller uses nftables through netlink and the NFQUEUE interface for packet delivery and verdicts. The queue is configured with a maximum length of 1,024, a copy length of 128 bytes, GSO support, and a 100-ms write timeout. These are implementation settings, not measured service rates or tail latency bounds. Limiting the copy to headers avoids copying whole payloads for policy decisions, but requires checking parser behavior for options, extension headers, fragmentation, and truncation.

The callback resolves policy through an evaluator interface and emits an acceptance conntrack label with the verdict. Metrics record callback processing time, protocol/family-specific verdict counts, queue occupancy, and drop counters. Callback time does not include every component of kernel-to-userspace waiting time, so it cannot alone represent application-visible connection setup. The benchmark harness did not scrape these agent metrics: its Prometheus queries cover node and kubelet CPU only, and a `kindnetd` CPU query in the saved configuration returned no samples. The implementation also has host-traffic and protocol exceptions; the conceptual cached path in Figure 1 is not a complete description of all hooks.

The repository builds several flavors of the agent. The standard flavor queues only traffic to or from local Pods selected by some policy, maintained as nftables sets of managed addresses. The IPTracker flavor inspected at the same revision instead reports *divert-all*: it queues every forwarded packet not already bypassed by conntrack, because it watches only NetworkPolicy objects and has no Pod informer from which to enumerate the local Pods a policy selects. The two flavors therefore have different λ_q for the same workload, and the queue-overload analysis of Section 3 applies to a larger traffic fraction in the IPTracker flavor.

The NRI resolver is an alternate source of local Pod metadata, not a replacement for every informer. Its callbacks add and remove local records; namespace information can still come from the API-derived view. This modularity permits policy logic to evolve as ordinary userspace code while the steering mechanism remains relatively stable. We regard reduced feature-development coupling as an architectural property, not a measured productivity result. New policy features may still require additional metadata, hook coverage, or cache invalidation.

An experimental distinction is essential: KNP names the inspected policy engine, while the benchmark directories label complete networking configurations as *Kindnet* or *Cilium*. Those configurations include networking and runtime components beyond policy evaluation. The benchmark repository pins Cilium v1.20.1 in its cluster specification and a `kube-network-policies:v1.1.0` image in its standalone install manifest, but the KNP cluster was created with the Kindnet CNI’s bundled policy agent, patched to `kindnet:v1.0.1` with an NRI socket mount; which policy binary and flags were effective in each Kindnet run is not recorded in the artifacts. We therefore do not attribute their

latency differences solely to the code described here.

6 Evaluation

6.1 Methodology and Provenance

We analyze archived ClusterLoader2 artifacts rather than execute a new cluster experiment. The repository describes a testbed with 720 `n2-standard-4` workers, a `c4-standard-96` control-plane node, and a target of 35,000 concurrent Pods. The runtime configuration is reported as containerd 2.2.4 with runc 1.3.5. These are environment descriptions, not a reconstructed per-run inventory; topology and binary provenance must be confirmed before pooling runs from different dates.

The extraction at benchmark revision `e8af592` inventories 1,809 artifact files and parses 1,285 JSON files, recovering 55 startup reports. Including diagnostic-only directories yields 62 attempts, seven without a startup report. Later revisions `0442dce` and `f998224` add descriptions of the raw-Pod abort and mesh-tier exclusion, respectively. Those records are used as documented outcomes, not as replacement timing samples. Appendix A identifies the relative source paths.

We distinguish `create_to_schedule`, `schedule_to_run`, and total `pod_startup`. The middle quantity includes runtime and container work; it is not a measurement of policy programming or first permitted communication. Component percentiles cannot be added to reconstruct a total percentile. Likewise, a configured object-operation QPS is not necessarily achieved Pod throughput: a ReplicaSet operation can create multiple Pods.

Saved configurations take precedence over the current generator. The September identity sweeps vary Pods per ReplicaSet along with intended identity cardinality, use the default scheduler, and replace 5,000 Pods per churn round. Five rounds within a run are not five independent trials. The number of allocated Cilium identities must also be distinguished from the generator’s intended label cardinality. We retain per-run percentiles and completion indicators, rather than constructing CDFs or confidence intervals from summary quantiles.

6.2 Workload Admission and Identity Turnover

Table 1 reports Cilium’s completed September identity tiers under three policy scenarios. Open gateway permits sandbox access to a gateway; hub-and-spoke adds selective gateway admission; mesh also permits sandbox peers in both directions. All target 35,000 Pods. Within the completed tiers, increasing intended identities does not produce an increasing post-scheduling tail. In the open-gateway case, P99 decreases from 5.930 s to 1.990 s between the endpoint tiers. The corresponding hub-and-spoke values are 6.370 s and 2.033 s.

Intended identities	Pods/RS	After-scheduling P99 (s)		
		Gateway	Spoke	Mesh
7	5,000	5.930	6.370	6.767
700	50	5.416	4.759	2.934
3,500	10	4.685	4.899	2.279
35,000	1	1.990	2.033	Not run

Table 1: Per-run archived Cilium startup percentiles. All eleven reported suites have zero JUnit failures. Identity counts are generator intentions; controller fan-out and burst size also change across rows.

Scenario	Identities	Pods/s P50 / P90	Node CPU (cores)	
			mean	P99
Gateway	7	284 / 377	0.52	0.71
	700	317 / 364	0.51	0.69
	3,500	316 / 372	0.54	0.69
	35,000	170 / 182	0.56	0.81
Spoke	7	0 [†] / 327	0.51	0.69
	700	323 / 398	0.61	0.86
	3,500	321 / 373	0.54	0.67
	35,000	169 / 181	0.55	0.78
Mesh	7	277 / 376	0.54	0.72
	700	302 / 351	0.54	0.67
	3,500	328 / 385	0.63	0.77
Kindnet QPS 500	~7	233 / 262	0.40	0.46
Kindnet tuned	~7	190 / 225	0.63	0.79

Table 2: Recovered Phase-1 scheduling throughput and worker-node CPU for the runs in Tables 1 and 3. CPU is the harness’s instantaneous node query on 4-vCPU workers, aggregated across nodes over the run; the Kindnet baseline row uses an earlier rate-based query and is not directly comparable. [†]More than half of the per-second samples were zero. No memory or policy-agent CPU series exist in the artifacts.

These results prevent a simple claim that identity cardinality alone causes Cilium startup collapse. They do not prove identity-independent scaling either: the generator changes controller fan-out and burst size with each tier. Table 2 makes the confound visible. Achieved Phase-1 scheduling throughput is 277–328 Pods/s (median of per-second samples) for every tier with 10 or more Pods per ReplicaSet, and 169–170 Pods/s for the two 35,000-ReplicaSet tiers; cluster-scoped Pod LIST P99 latency rises from 14–19 s to 28.8 s in the same tiers. The low post-scheduling P99 at 35,000 identities therefore coincides with a slower, controller-limited creation stream, which reduces the burst each node absorbs. One Pod per ReplicaSet is not equivalent to a few large ReplicaSets at the same object-operation rate. A controlled identity-turnover test should hold these quantities fixed and independently vary label sharing and label freshness.

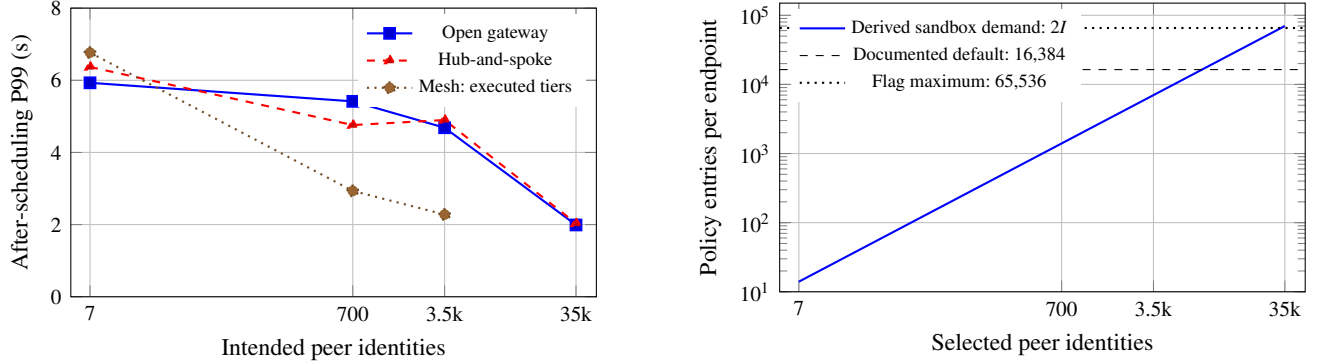


Figure 2: Different meanings of scaling. Left: measured per-run startup quantiles, with connecting lines only as visual guides. Right: conditional permission-state model, not measured occupancy. The 35k mesh tier was excluded and has no latency point. The default and maximum of `-bpf-policy-map-max` are from the v1.20.1 source; the setting effective in the benchmark is unrecorded.

Worker CPU does not track intended identity count in either family: Cilium’s mean stays within 0.51–0.63 cores and P99 within 0.67–0.86 across tiers and scenarios, and the tuned Kindnet run is in the same band. This is a whole-node aggregate that includes kubelet, runtime, and the sandbox workload; it cannot separate the policy agent’s share, and a per-Pod policy-map dump was not collected. What it does exclude is a CPU explosion at 35,000 identities on the nodes: if Cilium pays for identity cardinality in these configurations, it does so in the operator, the API server, or in map capacity, not in visible worker CPU.

The mesh tiers provide additional context. Post-scheduling medians remain approximately 1.41–1.42 s across the three executed tiers. Their end-to-end P99s are 33.534, 28.739, and 21.730 s, respectively, substantially above the post-scheduling values. Initial setup plus wait durations are 192.109, 137.947, and 109.718 s. These measurements are consistent with substantial work outside post-scheduling startup, but they do not causally isolate the responsible component.

Version matters as much as cardinality. An archived Cilium v1.18.6 attempt at the 700-identity gateway tier, run three hours before the v1.20.1 sweep on the same day, retains only its generated configuration and metadata: the harness never produced a startup report. The repository’s narrative records 34,981 Pods Running and 19 stranded in `Init:0/1` under default deny, and attributes the outcome to a policy-cache race between identity allocation and endpoint regeneration tracked as Cilium issue 7515 and resolved in v1.20.0 [1]. We have not reproduced that diagnosis from logs. The point for this paper is narrower: the same generator, at the same tier, produced an admission failure on one release and a clean 5.4-s P99 on the next. Identity-scaling claims about eager-materialization systems are release-specific, and a comparison must pin and report the version it tested.

There is no matched Kindnet identity or mesh sweep in the inspected artifacts. Every archived Kindnet run—the four

baseline QPS tiers and the five tuning bundles—uses six ReplicaSets of 5,000 Pods, that is, roughly seven intended identities. The repository’s summary table lists Kindnet values of 9.75, 9.80, and 9.85 s at 700, 3,500, and 35,000 identities, but no artifact directory backs those cells; they are not used here. We therefore plot no inferred constant Kindnet line, and an isolated tuned Kindnet run cannot establish invariance to identities, topology, or churn.

6.3 Capacity Exclusion Is Not a Latency Sample

The mesh experiment producer explicitly excluded the 35,000-identity tier. Its model assigns an entry for each sandbox identity in both directions at every endpoint. Equation 1 then gives approximately 70,000 sandbox entries per endpoint, before gateway, DNS, or other entries. This exceeds the stated 65,536-entry capacity. The runner lists only the three smaller tiers and contains a guard that skips estimated demands at or above the configured map limit.

This is meaningful design evidence: a system can have low latency throughout its feasible region and still encounter a state-capacity boundary. It is not evidence that an executed 35,000-identity mesh experienced insertion failures, packet loss, or a particular latency. We label it an *analytical exclusion*. The runner’s 65,536 figure is the maximum the agent accepts for `-bpf-policy-map-max`; the documented default is 16,384 entries per endpoint, which the runner uses as a fallback [14]. At the default, the same model excludes the 35,000-identity tier even for the unidirectional scenarios ($I = 35,000 > 16,384$), yet those tiers completed with 35,000 Pods Running. Either the tested configuration raised the limit, or the deployed policies did not expand per identity as the model assumes; the artifacts contain no `cilium-dbg bpf policy get dump` or `ConfigMap` to decide. The agent also exposes an `endpoint-lockdown-on-overflow` option [14], so

an executed overflow would surface as denied traffic rather than as a startup latency, which is one more reason the unrun tier has no P99.

Under the benchmark model, 3,500 selected identities and 50 local endpoints imply approximately 350,000 sandbox policy entries per node. This is derived demand, not a map dump or a memory measurement. The contrast with KNP is structural: its inspected design does not require this permission cross-product, but its capacity instead depends on metadata storage, connection state, and decision rate. A matched experiment remains necessary to compare those capacities in practice.

6.4 An Aborted Direct-Burst Run

The raw-Pod microsegmentation record documents a different outcome. An experiment creating Pods directly at the configured 500-QPS tier aborted with 29,306 Running and 5,694 stranded out of a 35,000-Pod target. The record attributes the abort to local Cilium REST API rate limiting during concurrent CNI ADD calls and exhaustion of the 65,280 allocatable cluster-local identities across creation and churn. It reports the allocator message “no more available IDs in configured space.” No final latency results were generated.

The missing percentile file is not a reason to erase a failed run. However, the documented diagnosis is not the same as a reconstructed event timeline. Allocator history, garbage collection, effective limiter settings, and the underlying logs are needed to separate the two causes and determine their interaction. Historical allocations can exceed the live-Pod count, but turnover alone does not imply monotonic consumption of an unreclaimable identity space.

The allocator namespace here is cluster-level state. It is distinct from the per-endpoint policy-map capacity used to exclude mesh tier 4. Likewise, direct Pod bursts differ from ReplicaSet-mediated creation. The raw-Pod abort supports an admission-failure case study, not an assertion that every 35,000-identity workload fails or that the unrun mesh tier has a measurable failure P99.

6.5 Connection Cost and Correctness

The available collection measures workload startup and associated control-plane activity, not the core packet-admission tradeoff. It does not establish NFQUEUE verdict capacity, first-connection latency, established-flow throughput parity, or isolation through startup and failure. A configured mesh permission does not create all-to-all traffic, and reaching Running does not test every forbidden peer relationship.

The only connection-level data point is the repository microbenchmark summarized in Section 4: 2,317 new connections/s through the queue on one node with sub-millisecond callback medians, and a P99 above 3 s once conntrack was exhausted [18]. Read against the startup sweeps, it says that a

node absorbing 50 sandboxes at 170–330 Pods/s cluster-wide is nowhere near the queue’s connection limit, and that the limit is reached first by connection-tracking capacity rather than by the userspace evaluator. It does not say what happens under sustained denied traffic, whether the P99 is queueing or conntrack, or how the numbers change with policy count.

The minimum discriminating extension is a small, controlled comparison of fresh and reused TCP/UDP flows at increasing offered rates, with permitted and denied traffic kept separate. It should report achieved connection rate, application-visible P50/P99 setup latency, queued packets per successful connection, agent CPU, queue drops, and conntrack occupancy. Repeated trials should include overload, rather than stop before the queue becomes the bottleneck, and should run once with fail-open and once fail-closed to measure the bypass window of Section 3. This can test the displaced cost without repeating a full 720-node sweep.

A companion lifecycle test should establish policy before Pod creation, then observe runtime metadata delivery and both allowed and forbidden connectivity from init containers onward. An NRI ablation needs otherwise identical versions and fairness settings. Policy changes, agent restart, queue overflow, and IP reuse are required to delimit the safety claim. These are remaining evaluation requirements, not experiments whose results this draft assumes.

6.6 Control-Plane Sensitivity and Limitations

The July QPS experiments illustrate why startup must be decomposed. Table 3 reports primary saved Cilium and Kindnet baseline runs. At QPS 500, their post-scheduling P99s are 5.482 s and 57.068 s, respectively. The archived baseline therefore does not establish a general startup advantage for the NFQUEUE-based configuration. At QPS 50 and 100, both families record two JUnit failures. All four are the same measurement, `WaitForOldPodsDeleted-Final`, timing out while 5,024 to 19,999 sandbox Pods from the churn phase were still terminating; no startup-phase measurement failed. The startup percentiles in those rows are therefore complete, but the runs did not finish their teardown within the harness deadline, and Pod deletion latency at these tiers is itself unmeasured.

The final Kindnet tuning bundle, labeled version 1.0.1 with NRI and APF exemption, records a post-scheduling P50 of 2.345 s and P99 of 9.718 s. Its total P99 is nevertheless 95.790 s, with a create-to-schedule P99 of 94.555 s. This juxtaposition is evidence against interpreting a reduced post-scheduling tail as an automatic end-to-end speedup. Versions, runtime integration, and API fairness settings changed together, so the sequence is not a causal NRI ablation. A separate Cilium QPS-500 repetition records 4.257 s post-scheduling P99 and 12.404 s total P99; it is retained as a separate trial.

API Priority and Fairness (APF) divides API-server concurrency among request classes and provides queueing and

QPS tier	After scheduling P99		Total P99	
	Cilium	Kindnet	Cilium	Kindnet
50*	4.065	4.833	4.089	4.855
100*	1.788	7.143	1.814	7.378
200	1.845	59.347	2.285	59.485
500	5.482	57.068	11.160	57.629

Table 3: Archived primary QPS runs; all values are seconds. Asterisks mark tiers whose only JUnit failures are final-teardown deletion timeouts in both configurations; startup measurements completed. These are complete-stack observations, not an isolated NFQUEUE versus eBPF comparison.

isolation [16]. Increasing one class’s share or exempting it can shift contention to other components; it does not create storage, CPU, or network capacity. The API server also mediates most declarative control-plane changes, including authorization, admission, persistence, and watch delivery. Its central role makes API latency relevant to network-policy evaluation, without implying that application packets pass through it.

The collection’s limitations affect the conclusions directly. Most configurations have too few independent trials for statistical claims. Summary percentiles hide time variation and exclude reconstruction of individual Pod trajectories. The counts of failed JUnit elements are not counts of failed Pods. CPU maxima based on instantaneous samples at the harness’s scrape interval are not true peaks. We use these artifacts to locate the questions the architecture must answer, not to manufacture a comparative scaling curve from missing measurements.

6.7 What the Artifacts Do Not Contain

A scan of both repositories for the measurements a reviewer would ask for produced the inventory in Table 4. Each row names the missing evidence, the claim it would support or refute, and the cheapest experiment that would produce it. The first row is the most important: without a KNP run at 700 or more intended identities, the paper’s central hypothesis—that deferred evaluation removes the identity dimension from the enforcement cost—is untested, and the flat Kindnet values in the repository’s summary table must be treated as unsupported until an artifact directory backs them.

7 Related Work

The closest precedent is reactive virtual switching, not another use of a kernel packet API. OVS combines userspace classification with kernel flow caches and discusses the performance consequences of that architecture [9]. NVP addresses distributed logical network realization [4]; OVN supports stateful ACLs [23]. Our proposed distinction is evaluation of Kubernetes policy semantics without eager per-endpoint

Missing evidence	Claim at stake	Cheapest producer
KNP at ≥ 700 identities (all runs use 6 RS)	identity-independent admission	rerun the three September generators on the Kindnet cluster
Policy-agent CPU and memory (only node CPU exists; kindnetd query had no samples)	where the displaced work lands	scrape agent metrics; fix the query’s container label
Queued-packet rate and verdict latency at 720 nodes (λ_q, μ)	queue-overload and fail-open window	enable agent /metrics scraping in the harness
Policy-map occupancy dumps and the effective map-size flag	capacity model of Fig. 2	cilium-dbg bpf policy get on one node per tier
Denied-traffic and first-connection tests	isolation and availability delay	single-node connection microbenchmark with deny mix
NRI on/off with identical binaries	startup-metadata benefit	rerun baseline QPS 500 and tuning bundle 1 with one pinned image
Effective binary and flags per Kindnet run	attribution to inspected code	record image digests and args in cl2-metadata
Raw logs for the v1.18.6 and raw-Pod aborts	causal diagnosis of both failures	retain agent and operator logs on abort

Table 4: Evidence absent from the benchmark and implementation repositories, found by scanning both at the cited revisions.

peer materialization, coupled to local runtime metadata. We do not claim the invention of userspace miss handling, cached verdicts, or stateful filtering.

Andromeda separates flow-processing paths to reconcile provisioning, isolation, performance, and feature velocity [2]. ONCache identifies repeated work across container-overlay layers and caches invariant results, including filtering, using eBPF [5]. These systems reinforce the importance of choosing what to cache and when to invalidate it. KNP focuses on the timing of semantic policy work, whereas ONCache accelerates repeated overlay processing. Its microbenchmark and application evaluations also illustrate the evidence missing from a startup-only comparison.

KUBEDIRECT identifies API-mediated controller communication as a serverless scaling bottleneck and develops state management for a more direct path [10]. It is relevant to our distinction between local runtime observation and eventual publication of state, but NRI-based address discovery does not provide KUBEDIRECT’s broader controller coordination. Role-based micro-segmentation infers endpoint roles and policies from traffic [6]; this addresses policy construction rather than enforcement of a given policy.

Finally, metadata aggregation is established practice. Cal-

ico Typha maintains datastore state, deduplicates events, and distributes updates to node agents to reduce their individual load on the datastore [25]. The IPTracker direction discussed next must therefore be evaluated on its representation, caching, and recovery tradeoffs, not on the novelty of inserting a distribution tier.

The eager-materialization model of Section 4 is one representation, not the only kernel-resident one. Open vSwitch’s conjunctive match (`conj_id`) lets a rule with n source and m destination members be installed as $n + m$ flows rather than $n \times m$ [22], and Antrea’s OVS pipeline uses exactly this to compile Kubernetes NetworkPolicy, marking established connections in `conntrack` so that only new flows traverse the policy tables [13]. Calico’s Felix compiles selectors into ipsets referenced by a bounded number of iptables or nftables rules, so per-endpoint state grows with the number of *rules* rather than peers [24]. Both approaches keep semantic evaluation in the kernel and avoid the per-peer cross-product by factoring the match; KNP avoids it by moving the match to userspace and caching the result. A fair capacity comparison must therefore include a factored-kernel baseline, not only the per-identity expansion that motivated the mesh-tier exclusion.

8 Discussion and Future Work

8.1 Compact Metadata Distribution with IPTracker

Avoiding a distributed security-identity allocator does not eliminate distributed metadata. With N agents each watching all Pod updates, an update stream of rate U entails approximately NU deliveries under an all-subscriber model. This is a first-order count, not a complete performance model; filtering, watch caching, event contents, and implementation details determine the actual cost.

IPTracker, introduced in the implementation’s PR 190, moves full Pod observation to an intermediate set of distributors and exposes compact IP-to-Pod records to agents [7]. If R distributors serve N nodes, the policy agents’ full Pod subscriptions against the API server can fall from N to R . Other subscriptions remain, and the distributor-to-agent fan-out does not disappear. In the inspected client, each agent watches the entire compact-record prefix. This is not selective subscription to only the peers an agent contacts.

The records are serialized with Protocol Buffers and may be stored locally in `bbolt`, with an in-memory LRU in front [17]. Kubernetes can itself use `protobuf`, so the relevant distinction is the reduced schema and distribution architecture, not a blanket JSON-versus-`protobuf` comparison. The disk store also records revision metadata for synchronization. A cache miss can consult local storage rather than perform a per-packet network lookup.

Locality offers a potential memory benefit: a node may communicate with a small fraction of the cluster’s endpoints

even when it stores metadata for all of them. However, a bounded object cache is not a bound on process or machine memory. `bbolt` mappings and page cache, full-sync response buffers, and whole-store listing contribute memory cost. The present LRU admits updates as well as lookup results, so metadata churn can evict frequently contacted peers. Store access under the wrapper’s mutex can also couple writes and lookup latency.

Recovery and freshness are equally important. The current full sync fetches the prefix, clears local state, and inserts records; this does not establish atomic snapshot replacement or constant-memory bootstrap. Revision recovery, compaction, reflector failover, disk failure, and IP reuse must be evaluated before persisted state is treated as current authorization metadata. Issue 114659 provides additional motivation for reducing informer memory costs, but it is not a quantitative demonstration of this implementation’s benefit [3].

The only quantitative data on IPTracker are the single-client integration-test logs posted with the repository’s scalability test (PR 218) [8]. With 10,000 records the server reported 35 MB of Go heap, 643 ms average propagation delay, and a 1 MB client store. A 40,000-record run at 30,000 writes/s reached 6.6 s propagation and 174 MB, and the `etcd` maintainer participating in the thread observed watch propagation stalling above that size. A 10-million record run throttled to 1,024 writes/s completed in 9,828 s with 43 ms average propagation, 24.7 GiB of server heap, and a 1.25 GiB client store. These are one process on one machine with no packet path, and the LRU in that test was sized to hold every record, so they bound neither agent memory under eviction nor the fan-out to hundreds of clients. They do show that the distribution tier, not the client, holds the bulk of the state, and that write rate rather than record count governs propagation delay in the tested range. IPTracker is implemented future work in this paper’s evaluation scope, not a feature credited with the reported startup results.

8.2 Testing the Architectural Hypothesis

The most informative next result is not necessarily a larger cluster. First, effective software versions, identity counts, map limits, and the source of each plotted result must be fixed. Existing lower-tier map dumps could validate the capacity model without executing a tier already excluded by the harness. Existing failure logs could establish the allocator and limiter timeline without reproducing an expensive abort.

Second, workload cardinality and workload freshness should be varied independently. A matched experiment can retain the same live-Pod count, generator, placement, offered rate, and policies while alternating between replacements that reuse label sets and replacements that introduce new ones. Measuring first permitted communication alongside startup and actual map occupancy would distinguish distribution and materialization from unrelated controller delay.

Third, the connection-cost experiment must determine when deferred evaluation stops being advantageous. If accepted-flow reuse is low, denied demand is high, or strict revocation repeatedly invalidates cached decisions, the userspace path may dominate. These outcomes would delimit the design’s useful region, not invalidate the use of NFQUEUE for every workload. Likewise, a demonstrated capacity benefit would not establish better packet throughput or immunity to control-plane overload.

8.3 Limits of the Present Claim

The design does not promise unlimited identities, constant total memory, zero startup delay, or topology-independent resource use. It avoids one form of eager permission expansion by retaining semantic interpretation in userspace. This is a useful separation only if the resulting admission and consistency costs are acceptable. The present evidence supports the mechanism and motivates the comparison, while leaving that quantitative boundary open.

9 Conclusion

Network-policy scalability is a joint problem of state representation, work placement, and synchronization. Fast kernel lookups do not guarantee cheap workload admission, and a flat latency curve does not imply unbounded policy capacity. NFQUEUE and conntrack provide an existing foundation for a different decomposition: interpret Kubernetes policy when communication is attempted and retain accepted connection decisions in the kernel. NRI can remove the asynchronous API round trip from local Pod-address discovery without pretending to eliminate all distributed-state dependencies.

The archived experiments show why this decomposition is worth evaluating carefully. Completed Cilium sweeps retain low post-scheduling latency and flat worker CPU while the 35,000-ReplicaSet tiers halve achieved throughput; the producer excludes a dense-mesh tier on predicted permission-state capacity grounds; and two admission failures—one release-specific, one allocator- and limiter-specific—are recorded. Together they motivate a comparison based on feasible state and attempted communication, not a universal ranking of datapath technologies. Establishing the benefit of deferred policy evaluation now requires the eight measurements of Table 4, starting with a matched KNP identity sweep and an explicit test of the fail-open overload window.

References

- [1] Cilium contributors. Pods stuck in Init when policy map is full. Cilium issue 7515, closed by pull request 44900. <https://github.com/cilium/cilium/issues/7515>. Accessed September 17, 2026; the fix is reported in the v1.20.0 release notes.
- [2] Michael Dalton, David Schultz, Jacob Adriaens, Ahsan Arefin, Anshuman Gupta, Brian Fahs, Dima Rubinstein, Enrique Cauich Zermeno, Erik Rubow, James Alexander Docauer, Jesse Alpert, Jing Ai, Jon Olson, Kevin DeCaboooter, Marc de Kruijf, Nan Hua, Nathan Lewis, Nikhil Kasinadhuni, Riccardo Crepaldi, Srinivas Krishnan, Subbaiah Venkata, Yossi Richter, Uday Naik, and Amin Vahdat. Andromeda: Performance, isolation, and velocity at scale in cloud network virtualization. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*. USENIX Association, 2018. <https://www.usenix.org/conference/nsdi18/presentation/dalton>.
- [3] jeevanragula and Kubernetes contributors. Unable to set external store in client-go. Kubernetes issue 114659, 2022. <https://github.com/kubernetes/kubernetes/issues/114659>. Opened December 22, 2022; accessed September 16, 2026.
- [4] Teemu Koponen, Keith Amidon, Peter Balland, Martín Casado, Anupam Chanda, Bryan Fulton, Igor Ganichev, Jesse Gross, Natasha Gude, Paul Ingram, Ethan Jackson, Andrew Lambeth, Romain Lenglet, Shih-Hao Li, Amar Padmanabhan, Justin Pettit, Ben Pfaff, Rajiv Ramanathan, Scott Shenker, Alan Shieh, Jeremy Stribling, Pankaj Thakkar, Dan Wendlandt, Alexander Yip, and Ronghua Zhang. Network virtualization in multi-tenant datacenters. In *11th USENIX Symposium on Networked Systems Design and Implementation (NSDI 14)*. USENIX Association, 2014. <https://www.usenix.org/conference/nsdi14/technical-sessions/presentation/koponen>.
- [5] Shengkai Lin, Shizhen Zhao, Peirui Cao, Xinchu Han, Quan Tian, Wenfeng Liu, Qi Wu, Donghai Han, and Xinbing Wang. ONCache: A cache-based low-overhead container overlay network. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. USENIX Association, 2025. <https://www.usenix.org/conference/nsdi25/presentation/lin-shengkai>.
- [6] Sathiya Kumaran Mani, Kevin Hsieh, Santiago Segarra, Ranveer Chandra, Yajie Zhou, and Srikanth Kandula. Securing public cloud networks with efficient role-based micro-segmentation. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. USENIX Association, 2025. <https://www.usenix.org/conference/nsdi25/presentation/mani>.
- [7] Antonio Ojea. Optimize pod IP and metadata distribution for large clusters using kube-ip-tracker. kube-network-policies pull request 190, 2025. <https://github.com/kubernetes-sigs/kube-network-policies/pull/190>. Accessed September 16, 2026; implementation history, not an independent evaluation.
- [8] Antonio Ojea and Marek Siarkowicz. Add scalability tests for ipcache. kube-network-policies pull request 218, discussion thread, 2025. <https://github.com/kubernetes-sigs/kube-network-policies/pull/218>. Accessed September 17, 2026; single-client integration test logs, not an independent evaluation.
- [9] Ben Pfaff, Justin Pettit, Teemu Koponen, Ethan Jackson, Andy Zhou, Jarno Rajahalme, Jesse Gross, Alex Wang, Joe Stringer, Pravin Shelar, Keith Amidon, and Martín Casado. The design and implementation of Open vSwitch. In *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*. USENIX Association, 2015. <https://www.usenix.org/conference/nsdi15/technical-sessions/presentation/pfaff>.
- [10] Sheng Qi, Zhiqian Zhang, Xuanzhe Liu, and Xin Jin. KUBEDIRECT: Unleashing the full power of the cluster manager for serverless computing. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*. USENIX Association, 2026. <https://www.usenix.org/conference/nsdi26/presentation/qi>.
- [11] spikecurtis and Kubernetes contributors. Pod.Status.PodIP not updated during postStart lifecycle hook. Kubernetes issue 85966, 2019. <https://github.com/kubernetes/kubernetes/issues/85966>. Opened December 5, 2019; closed January 30, 2025; accessed September 16, 2026.
- [12] Phil Sutter. Benchmarking nftables. Red Hat Developer, 2017. <https://developers.redhat.com/blog/2017/04/11/benchmarking-nftables>. Published April 11, 2017; accessed September 16, 2026.
- [13] The Antrea Authors. Antrea OVS pipeline. Antrea design documentation. <https://antrea.io/docs/main/docs/design/ovs-pipeline/>. Accessed September 17, 2026; rolling documentation.
- [14] The Cilium Authors. cilium-agent command reference. Cilium documentation, `-bpf-policy-map-max`.

- <https://docs.cilium.io/en/stable/cmdref/cilium-agent/>. Accessed September 17, 2026. The v1.20.1 source (pkg/maps/policymap/cell.go, pkg/option/config.go) sets the default to 16,384 and clamps the flag to [256, 65,536]; neither is a record of the benchmark configuration.
- [15] The Cilium Authors. Security identities. Cilium documentation. <https://docs.cilium.io/en/stable/internals/security-identities/>. Accessed September 16, 2026; not a benchmark version record.
 - [16] The Kubernetes Authors. API priority and fairness. Kubernetes documentation. <https://kubernetes.io/docs/concepts/cluster-administration/flow-control/>. Accessed September 16, 2026.
 - [17] The Kubernetes Authors. kube-network-policies: BoltStore implementation. Source code, pkg/ipcache/bbolt.go. <https://github.com/kubernetes-sigs/kube-network-policies/blob/main/pkg/ipcache/bbolt.go>. Local source inspected September 16, 2026; verify pinned commit accessibility before publication.
 - [18] The Kubernetes Authors. kube-network-policies testing notes: ApacheBench microbenchmark. Repository documentation, docs/testing/README.md. <https://github.com/kubernetes-sigs/kube-network-policies/blob/main/docs/testing/README.md>. Local copy inspected at revision 9984dcd; hardware and software versions are not recorded.
 - [19] The Kubernetes Authors. Network policies. Kubernetes documentation. <https://kubernetes.io/docs/concepts/services-networking/network-policies/>. Accessed September 16, 2026; see Pod lifecycle and existing connections.
 - [20] The Linux Kernel Contributors. BPF maps. Linux kernel documentation. <https://docs.kernel.org/bpf/maps.html>. Accessed September 16, 2026.
 - [21] The Netfilter Project. libnetfilter_queue: Queue handling. API documentation, version 1.0.5. https://netfilter.org/projects/libnetfilter_queue/doxygen/html/group__Queue.html. Accessed September 16, 2026.
 - [22] The Open vSwitch Project. ovs-fields(7): Conjunctive match fields. Open vSwitch manual, conj_id. <https://www.openvswitch.org/support/dist-docs/ovs-fields.7.txt>. Accessed September 17, 2026.
 - [23] The OVN Project. ovn-nb(5): OVN north-bound database schema. OVN manual, ACL table. <https://www.ovn.org/support/dist-docs/ovn-nb.5.html>. Accessed September 16, 2026; rolling documentation.
 - [24] Tigera. Component architecture. Calico Open Source documentation, Felix and Typha. <https://docs.tigera.io/calico/latest/reference/architecture/overview>. Accessed September 17, 2026.
 - [25] Tigera. Typha overview. Calico Open Source documentation. <https://docs.tigera.io/calico/latest/reference/typha/overview>. Accessed September 16, 2026.
 - [26] Dan Winship. NFTables mode for kube-proxy. Kubernetes Blog, 2025. <https://kubernetes.io/blog/2025/02/28/nftables-kube-proxy/>. Published February 28, 2025; accessed September 16, 2026.

A Evidence Provenance

This appendix is an internal provenance guide for the non-submission exercise. Paths are relative to the benchmark repository unless stated otherwise. It is not anonymized publication material.

Archived numeric results. Benchmark revision e8af592 is the source snapshot for the numeric inventory. The extractor is `submission/analyze.mjs`; the current extraction is recorded in `submission/upstream/inventory.json`, `submission/upstream/startup.csv`, and `submission/upstream/extracted-results.md`. Run names do not independently establish tested software versions.

Table 1 and the measured panel of Figure 2 use reports beneath `artifacts_pods/cilium/identity-sweep/`, `artifacts_pods/cilium/microsegmentation/`, and `artifacts_pods/cilium/mesh-sweep/`. The mesh reports are in `tier1-7-identities-500ppr`, `tier2-700-identities-50ppr`, and `tier3-3500-identities-10ppr`. Their sibling `*-initial` directories contain hook logs, not startup reports.

Table 3 uses the primary runs under `artifacts_pods/cilium/qps-*`, including `qps-500/verified-run`, and `artifacts_pods/kindnet/baseline-sweep/qps-*`. The separate Cilium trial is in `qps-500/repeat-run`. The final Kindnet tuning bundle is under `artifacts_pods/kindnet/500qps-optimizations/5-v1.0.1-plus-nri-plus-apf-exempt/`. The teardown-timeout diagnosis of the asterisked rows is from the `junit.xml` files in the QPS 50 and 100 directories of both families.

Recovered throughput, CPU, and API latency. Table 2 reads `Phase1SchedulingThroughput_*.json` (`perc50`, `perc90`) and `GenericPrometheusQueryWorkerNodeCPUUtilization_*.json` (`mean_`, `p99_instantaneous_node_cpu_cores`) in each run directory; the Kindnet baseline uses `mean_node` and `p99_node` from an earlier query. Pod LIST latency is the cluster-scoped `Pods/LIST` row of `APIResponsivenessPrometheus_simple_*.json`. The hub-and-spoke rows come from `artifacts_pods/cilium/microsegmentation/tier*`. The extraction is reproducible with a short script over these files; no value was transcribed from the repository’s Markdown summary.

Archived v1.18.6 attempt. `artifacts_pods/archive/v1.18.6-identity-runs/` holds a complete 7-identity run and a 700-identity directory containing only `cl2-metadata.json` and the generated configuration. The 34,981/19 counts and the issue-7515 attribution are from `benchmark_results.md`, not from artifacts.

Outcome records and capacity model. Revision 0442dce adds `artifacts_pods/cilium/microsegmentation/tier4-35000-identities-rawpods/README.md`, the source of the 29,306/5,694 completion counts and stated diagnosis. Revision f998224 adds `artifacts_pods/cilium/mesh-sweep/README.md`, documenting the unexecuted mesh tier. The model and exclusion are cross-checked against `scripts/run-mesh-sweep.sh` and `manifests/policy-scenarios/scenario-c-mesh-bidirectional.yaml`. These sources do not replace verification of the tested Cilium map representation and effective configuration.

Original charts. The input figure `figures/fig2_identity_cardinality_sweep.png` mixes measured quantiles, unmatched Kindnet tier values, and a failure bar. The separate `figures/fig2_topological_scaling_p99.png` contains a 94.3-second overflow value and a kernel-set description of KNP that the audited evidence does not support. Both originals are preserved; neither is reproduced as a verified result here. Figure 2 is typeset directly from the audited quantiles and the explicitly conditional capacity model.

Implementation. Paths in the companion kube-network-policies repository are `pkg/dataplane/controller.go` (queue configuration, bypass flag, postrouting and input chains), `pkg/cmd/cmd.go` (`-fail-open` default), `pkg/networkpolicy/networkpolicy.go` (`nil-peer` selector semantics, managed-IP set), `plugins/iptracker/iptracker_networkpolicy.go` (`divert-all`), `pkg/podinfo/nri/resolver.go`, `pkg/ipcache/{client,bbolt,lru}.go`, and `docs/testing/README.md` with its two PNG charts, inspected at 9984dcd. Benchmark-side deployment facts are from `manifests/kube-network-policies-install.yaml` and `scripts/create-netpol.sh`. Source behavior has not been equated to the benchmark binaries. `submission/evidence-audit.md` records the claim-level limitations, and `submission/author-guide.md` provides the prioritized remaining checks.